

Stochastic Anomaly Simulator

Project Documentation

Pole Anomaly Project

May 12, 2026

Contents

1	Introduction	3
2	Quick Setup Guide	4
2.1	Environment Setup	4
2.2	Dependencies	4
3	Synthetic Data Generation Pipeline (data-gen)	5
3.1	File Structure Review	5
3.2	Explanation of the Engine and Scripts	5
3.2.1	data-gen/src/anomaly_generator.py	5
3.2.2	data-gen/script/generate_custom_dataset.py	5
3.2.3	data-gen/script/test_generator_deformation.py	6
3.2.4	data-gen/script/visualize_anomalies_templates.py	6
3.3	Anomaly Template Creation	6
3.3.1	Configuration Blueprint (CSV Logic)	6
4	Neural Networks: Theory and Fundamentals	9
4.1	The Artificial Neuron	9
4.1.1	Activation Functions	9
4.2	From Neurons to Networks	9
4.3	The Loss Function	10
4.4	Gradient Descent and Backpropagation	10
4.4.1	Gradient Descent	10
4.4.2	Backpropagation	10
4.4.3	Stochastic Gradient Descent and Adam	11
4.5	Regularization: Preventing Overfitting	11
4.6	Convolutional Neural Networks (CNN)	11
4.6.1	The Convolution Operation	11
4.6.2	Pooling	12
4.6.3	Why 1D-CNN for Anomaly Detection?	12

5	1D-CNN Implementation	13
5.1	Data Processing (<code>train.py</code>)	13
5.2	Architecture Definition (<code>model.py</code>)	13
5.3	Training Loop (<code>train.py</code>)	14
5.3.1	Training Results	14
5.4	Architecture Search	15
5.5	Inference and Evaluation (<code>visualize_inference.py</code>)	16
5.5.1	Inference Results	16
5.6	Robustness Analysis (<code>robustness_test.py</code>)	17

1 Introduction

This document provides a comprehensive overview of the **Pole Anomaly Project**, an anomaly detection and classification system for nuclear gamma dosimetry signals. The overall objective is to automatically identify and classify transient anomalous events—such as radiation spikes with distinct geometric signatures—embedded within continuous, noisy gamma dose rate measurements.

The project follows a two-phase approach. First, a **synthetic data generation pipeline** produces large-scale training datasets by injecting user-defined anomaly templates (Bell curves, Squares, M-shapes, Exponential ramps, etc.) into realistic simulated radiation backgrounds derived from real-world sensor baselines. Second, a **1D Convolutional Neural Network (1D-CNN)** is trained on these synthetic datasets to perform multi-class anomaly classification in a streaming inference setting.

The repository is organized into three main components:

- **data-gen/**: The synthetic data generation pipeline, including anomaly templates, the stochastic generator engine, and visualization scripts.
- **1D-CNN/**: The neural network implementation, including the model architecture, training loop, inference engine, and robustness testing scripts.
- **documentation/**: This document and associated media.

2 Quick Setup Guide

This project isolates dependencies using a Python virtual environment to ensure absolute reproducibility.

2.1 Environment Setup

To initialize the project on a new machine, execute the following commands in the root of the project:

```
# 1. Create a Python Virtual Environment
py -3.14 -m venv .venv

# 2. Activate the Environment (Windows PowerShell)
.\.venv\Scripts\Activate.ps1

# 3. Install all dependencies
pip install -r requirements.txt
```

2.2 Dependencies

The `requirements.txt` installs the following libraries:

- `numpy` & `pandas`: Fast numerical array manipulation and CSV template handling.
- `torch` (PyTorch): Deep learning framework for the 1D-CNN. Installed from the PyTorch Nightly channel with CUDA 12.8 support via `--extra-index-url`.
- `plotly` & `kaleido`: Interactive HTML visualizations and static PNG export.
- `scikit-learn`: Utility functions for metrics and data splitting.
- `matplotlib`: Supplementary plotting library.
- `scipy`, `statsmodels`, `tslearn`: Statistical interpolation and legacy dataset compatibility.

Note: The `--pre` flag in `requirements.txt` enables installation of pre-release PyTorch builds, which is required for CUDA 12.8 support on newer GPUs (e.g., NVIDIA RTX 5060).

3 Synthetic Data Generation Pipeline (data-gen)

As the repository expands to include other projects (such as neural networks for anomaly detection), this section specifically details the anomaly generation pipeline.

3.1 File Structure Review

The repository enforces a strict separation of concerns, divided into modular directories:

- `data-gen/data/`: Contains raw background sensor readings (e.g., `2015_months_DebitDoseA.txt`) used as a real-world baseline, along with generated output datasets and the class mapping file (`class_mapping.txt`).
- `data-gen/anomalies/`: Contains the normalized `.csv` blueprints (templates) for the 6 anomaly shapes: `bell`, `bell_sq`, `fast_ascend`, `fast_descend`, `M`, and `square`.
- `data-gen/src/`: Stores the pure mathematical generator library (`anomaly_generator.py`), decoupled from direct script logic.
- `data-gen/script/`: Stores execution pipelines and visualization launchers. Scripts utilize the math from `data-gen/src/` to generate datasets or debug plots.
- `data-gen/logs/`: The destination for Plotly’s interactive HTML visualizations and PNG exports.

3.2 Explanation of the Engine and Scripts

3.2.1 `data-gen/src/anomaly_generator.py`

This is the core computational engine. It provides the `generate_anomaly()` function which transforms a normalized CSV blueprint into a physical discrete array. The engine mathematically scales the template temporally (period span) and vertically (amplitude intensity) while embedding Gaussian deformation using the limits declared in the CSV file.

3.2.2 `data-gen/script/generate_custom_dataset.py`

The primary execution pipeline for training data. It analyzes the original physical gamma sensor data using moving averages to dynamically separate high-frequency point-to-point noise from the slow, low-frequency sensor drift. It then simulates a highly realistic **wandering baseline** using a chunk-by-chunk Ornstein-Uhlenbeck (OU) mean-reverting random walk process, overlaying the high-frequency Gaussian noise on top.

Spanning datasets of up to 10,000,000 steps, it randomly samples anomaly templates from the library, calculates randomized injection amplitudes and duration bounds, and seamlessly injects them into the wandering baseline. It maps each injected shape to a unique multi-class integer ID, tagging these absolute ground-truth labels alongside the signal.

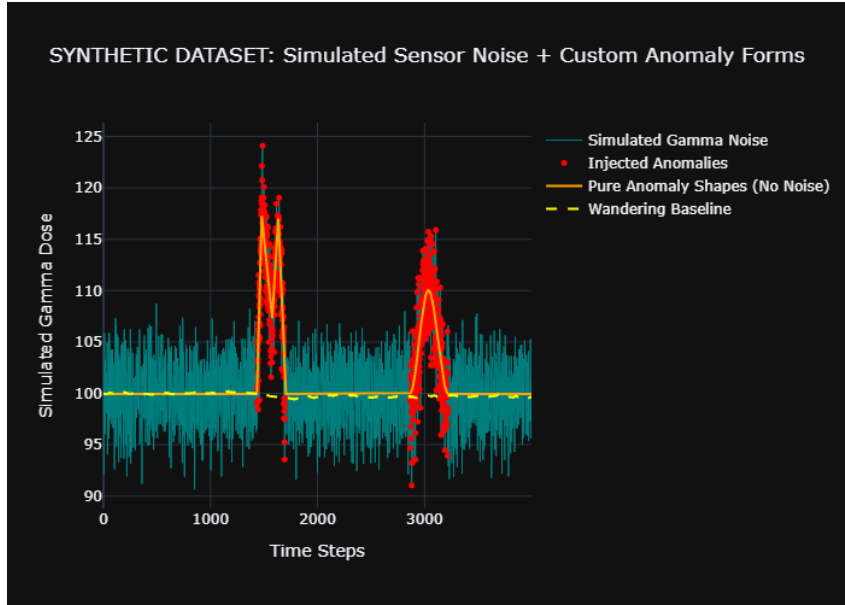


Figure 1: Output of `generate_custom_dataset.py`

3.2.3 `data-gen/script/test_generator_deformation.py`

A visual sandbox and boundary debugging tool. Iterating automatically over all discovered templates in the `data-gen/anomalies/` directory, it runs exactly 5 generation iterations of each template to visually verify and validate the variability limits enforced by the template's deformation parameters. Exports directly to HTML and PNG.

3.2.4 `data-gen/script/visualize_anomalies_templates.py`

This script visualizes the pure original mathematical blueprints. It renders the shapes and their interpolation properties exactly as described in the blueprint CSV, without injecting randomization or temporal noise.

3.3 Anomaly Template Creation

Instead of relying on rigid pre-recorded dataset distributions, targeted geometric structures are freely defined as CSV blueprints in the `data-gen/anomalies/` folder.

3.3.1 Configuration Blueprint (CSV Logic)

A standard anomaly template requires 7 strict column definitions mapping normalized coordinates and behavioral constraints:

`time, value, t_minus, t_plus, v_minus, v_plus, interp`

- **time & value:** Normalized shape coordinates (scaling from 0.0 to 1.0). They define the key inflection points of the curve.
- **Deformation Multipliers (`t_minus`, `t_plus`, `v_minus`, `v_plus`):** Define spatial jitter limits per control point. A value of 1.0 allows symmetric Gaussian distortion up to 100% of the global variance parameter. A value of 0.0 locks the point rigidly in that direction.

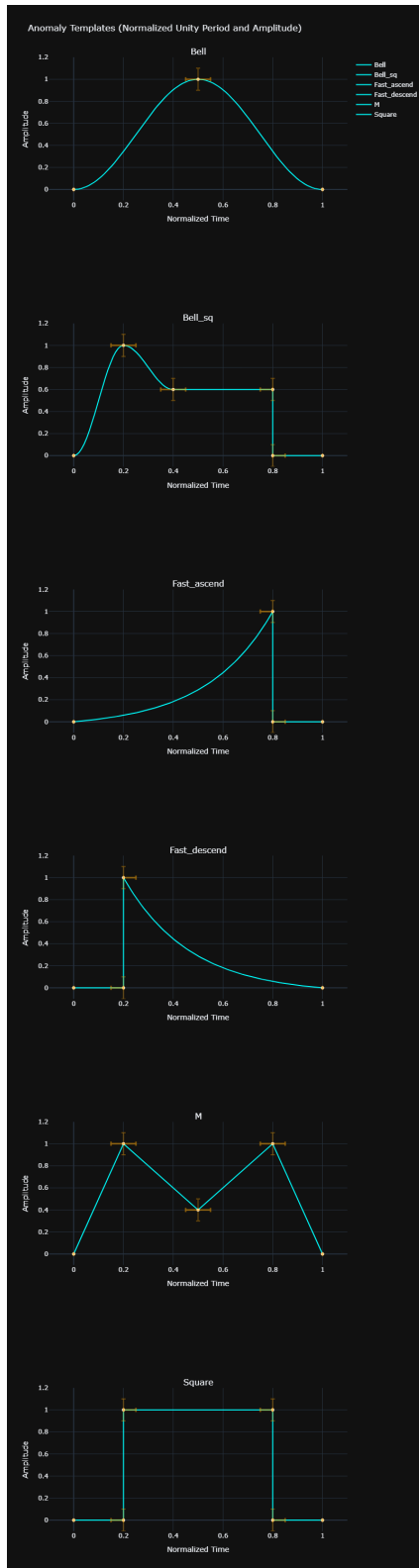


Figure 2: Output of visualize_anomalies_templates.py

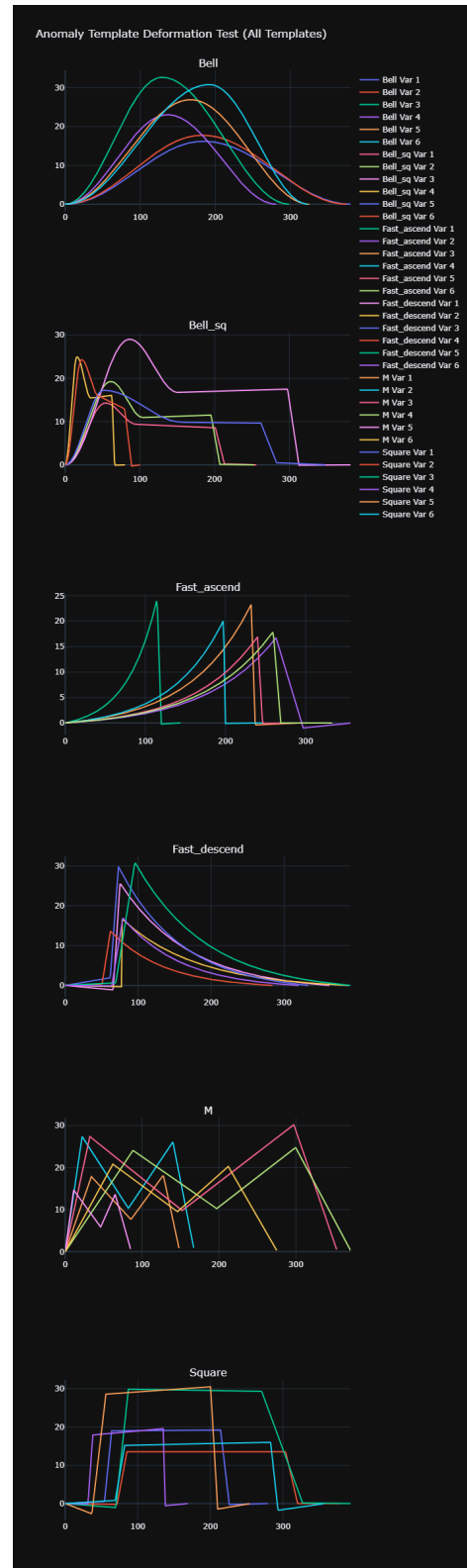


Figure 3: Output of test_generator_deformation.py

- **interp (Interpolation Mode):** Defines the mathematical connection between consecutive control points:
 - **linear:** Simple linear interpolation between points.
 - **exp_up:** Concave-upward exponential growth (slow start, rapid finish), with curvature controlled by $\alpha = 3.0$.
 - **exp_down:** Concave-downward exponential curve (rapid start, slow finish), the complement of **exp_up**.
 - **bell:** Cosine envelope $(\frac{1}{2} - \frac{1}{2} \cos(\pi x))$, generating a smooth S-curve suitable for Gaussian-like bell shapes.

4 Neural Networks: Theory and Fundamentals

This section provides a self-contained introduction to neural networks, building the theoretical foundation necessary to understand the 1D-CNN classifier implemented in this project. No prior knowledge of machine learning is assumed.

4.1 The Artificial Neuron

A neural network is built from simple computational units called **neurons** (or **perceptrons**). Each neuron takes a vector of inputs $\mathbf{x} = [x_1, x_2, \dots, x_n]$, multiplies each input by a learned **weight** w_i , adds a **bias** term b , and passes the result through a non-linear **activation function** σ :

$$y = \sigma\left(\sum_{i=1}^n w_i x_i + b\right) = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (1)$$

The weights and bias are the **learnable parameters** of the neuron. During training, these values are adjusted so that the neuron's output y matches the desired output for a given input.

4.1.1 Activation Functions

Without the non-linear activation function σ , stacking multiple neurons would be equivalent to a single linear transformation—no matter how many layers are used, the overall mapping would remain linear. Common activation functions include:

- **ReLU** (Rectified Linear Unit): $\sigma(z) = \max(0, z)$. The most widely used activation in modern networks. It outputs zero for negative inputs and passes positive values unchanged. This introduces non-linearity while avoiding the vanishing gradient problem that plagued earlier functions like Sigmoid.
- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}}$. Squashes the output to the range $(0, 1)$, useful for binary probability outputs.
- **Softmax**: Applied to a vector of logits $\mathbf{z}$, it converts them into a probability distribution: $\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$. Used in the output layer for multi-class classification to ensure the predicted probabilities sum to 1.

4.2 From Neurons to Networks

A **neural network** is formed by organizing neurons into **layers**:

- **Input layer**: Receives the raw data (e.g., a 512-point signal window).
- **Hidden layers**: Intermediate layers that progressively transform the data. Each layer extracts increasingly abstract features from its input. For example, the first hidden layer might detect simple edges, while deeper layers combine those edges into complex shapes.
- **Output layer**: Produces the final prediction (e.g., 7 class probabilities for our anomaly classifier).

When every neuron in one layer is connected to every neuron in the next, the layer is called a **fully connected** (or **dense**) layer. The full computation from input to output is called the **forward pass**.

4.3 The Loss Function

To train the network, we need a mathematical measure of how wrong its predictions are. This is the **loss function** $\mathcal{L}$, which compares the network's output $\hat{y}$ to the true label y . The goal of training is to minimize this loss.

For multi-class classification, the standard loss function is **Cross-Entropy Loss**:

$$\mathcal{L} = - \sum_{c=1}^C y_c \log(\hat{y}_c) \quad (2)$$

where C is the number of classes, y_c is 1 for the true class and 0 otherwise (one-hot encoding), and $\hat{y}_c$ is the predicted probability for class c . This loss is zero when the prediction is perfect and increases sharply when the model assigns low probability to the correct class.

4.4 Gradient Descent and Backpropagation

Training a neural network means finding the values of all weights $\mathbf{w}$ and biases b that minimize the loss $\mathcal{L}$.

4.4.1 Gradient Descent

Imagine the loss function as a high-dimensional landscape where the height at each point represents the error for a particular configuration of weights. Gradient descent works by:

1. Computing the **gradient** $\nabla_{\mathbf{w}}\mathcal{L}$ —the vector of partial derivatives of the loss with respect to each weight. This points in the direction of steepest increase.
2. Taking a step in the **opposite** direction (downhill) to reduce the loss:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla_{\mathbf{w}}\mathcal{L} \quad (3)$$

where η is the **learning rate**, a hyperparameter controlling the step size. Too large and the optimizer overshoots the minimum; too small and convergence is impractically slow.

4.4.2 Backpropagation

Computing the gradient of the loss with respect to every weight in a deep network is made efficient by the **backpropagation algorithm**. It applies the chain rule of calculus layer by layer, propagating the error signal backward from the output layer to the input layer:

For a weight w in layer l that produces output $z_l = w \cdot a_{l-1} + b$ (where a_{l-1} is the activation from the previous layer), the gradient is decomposed as:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z_l} \cdot \frac{\partial z_l}{\partial w} = \delta_l \cdot a_{l-1} \quad (4)$$

where $\delta_l = \frac{\partial \mathcal{L}}{\partial z_l}$ is the **error signal** at layer l . The key insight is that δ_l can be computed recursively from δ_{l+1} , making the computation scale linearly with network depth rather than exponentially.

4.4.3 Stochastic Gradient Descent and Adam

In practice, computing the gradient over the entire dataset is prohibitively expensive. **Stochastic Gradient Descent (SGD)** approximates the true gradient using a small random subset (**mini-batch**) of the data. The **Adam** optimizer (used in this project) extends SGD by maintaining adaptive per-parameter learning rates using exponential moving averages of the first and second moments of the gradients, leading to faster and more stable convergence.

4.5 Regularization: Preventing Overfitting

A model that memorizes the training data but fails to generalize to unseen data is said to **overfit**. Several techniques prevent this:

- **Dropout:** During training, a random fraction of neuron activations are set to zero at each forward pass (e.g., 40% dropout). This forces the network to learn redundant representations rather than relying on any single neuron, acting as an implicit ensemble of smaller networks.
- **Batch Normalization:** Normalizes the input to each layer by subtracting the batch mean and dividing by the batch standard deviation. This stabilizes the distribution of activations throughout training, enabling higher learning rates and faster convergence.
- **Early Stopping:** Monitor the validation loss during training. When it stops improving for a set number of epochs (**patience**), training is halted and the best-performing weights are restored. This prevents the model from continuing to memorize noise in the training set.

4.6 Convolutional Neural Networks (CNN)

Standard fully connected networks treat each input element independently, ignoring spatial or temporal relationships. **Convolutional Neural Networks** address this by using **convolutional layers**, which slide a small learnable filter (or **kernel**) across the input to detect local patterns.

4.6.1 The Convolution Operation

For a 1D input signal $x[t]$ and a kernel k of size K , the convolution produces a feature map:

$$(x * k)[t] = \sum_{i=0}^{K-1} k[i] \cdot x[t+i] \quad (5)$$

This operation has three key properties:

1. **Local connectivity:** Each output element depends only on a small neighborhood of the input (the receptive field), not the entire signal.

2. **Weight sharing:** The same kernel is applied at every position along the signal, drastically reducing the number of parameters compared to a fully connected layer.
3. **Translation equivariance:** If a pattern shifts in time, the corresponding activation in the feature map shifts by the same amount. The network detects the pattern regardless of where it appears.

4.6.2 Pooling

After convolution, **pooling layers** downsample the feature maps by taking the maximum (**MaxPooling**) or average (**AvgPooling**) over non-overlapping regions:

$$\text{MaxPool}(x)[t] = \max_{i \in [2t, 2t+1]} x[i] \quad (6)$$

Pooling reduces the spatial resolution, lowers computational cost, and increases the effective receptive field of subsequent layers. For a window of 512 points, three MaxPool layers with kernel 2 reduce the sequence from $512 \rightarrow 256 \rightarrow 128 \rightarrow 64$ points.

4.6.3 Why 1D-CNN for Anomaly Detection?

In this project, the input is a **1D time series** (gamma dose rate vs. time), not a 2D image. A 1D-CNN is the natural choice because:

- Anomaly shapes (Bell, Square, M, etc.) are defined by their **local temporal morphology**—the specific sequence of rises, plateaus, and falls. 1D convolutions are designed to detect exactly these local patterns.
- The **translation equivariance** of convolutions means the network can detect an anomaly regardless of where it appears in the signal window.
- Compared to recurrent networks (RNNs/LSTMs), CNNs are **highly parallelizable**, making them significantly faster for both training and inference on GPU hardware.
- The small kernel sizes (5–7 points) match the temporal scale of the anomaly features, and stacking multiple convolutional blocks builds up the receptive field to capture the full anomaly structure (80–400 points).

5 1D-CNN Implementation

This section describes the concrete PyTorch implementation of the 1D-CNN for multi-class anomaly classification. The codebase is organized into modular scripts within the `1D-CNN/` directory, with full CUDA GPU acceleration support.

5.1 Data Processing (`train.py`)

The data loading and preprocessing logic is embedded in the `load_data()` function within `train.py`. It bridges the synthetic `.csv` generator and the neural network through the following steps:

1. **Temporal Split:** The continuous signal is split chronologically *before* windowing: the first 80% of time steps are reserved for training and the remaining 20% for testing. This prevents **data leakage**, which would occur with a random split because overlapping windows (stride 32 over a 512-point window) share up to 94% of their data points.
2. **Sliding Window Extraction:** Each split is independently sliced into overlapping windows (size 512, stride 32).
3. **Threshold-Based Labeling:** A window is only labeled as a specific anomaly class if the anomaly occupies at least **10%** of the window length. This prevents the CNN from learning that windows containing 99% pure noise should be classified as anomalies. When the threshold is met, the most frequent anomaly class ID within the window is assigned via majority voting.
4. **Per-Window Centering (Instance Normalization):** To make the network robust against the slowly wandering background baseline generated by the Ornstein-Uhlenbeck process, each sliding window is independently centered by subtracting its own local mean μ_{window} . However, a global standard deviation σ_{train} is computed across all centered training windows to preserve the relative amplitudes of the spikes. The test set is scaled using this same global statistic. This value is persisted to `normalization_stats.json` for identical inference normalization:

$$x_{\text{scaled}} = \frac{x - \mu_{\text{window}}}{\sigma_{\text{train}}} \quad (7)$$

5. **Tensor Reshaping:** The output is reshaped into the strict `(Batch_Size, 1, Window_Size)` format required by PyTorch’s `Conv1d` layers.

5.2 Architecture Definition (`model.py`)

The `Anomaly1DCNN` class inherits from `torch.nn.Module` and implements a 3-block convolutional architecture with a progressively narrowing kernel strategy:

- **Block 1:** `Conv1d(in=1, out=16, kernel=7, padding=3) → BatchNorm1d → ReLU → MaxPool1d(2)`.
- **Block 2:** `Conv1d(in=16, out=32, kernel=7, padding=3) → BatchNorm1d → ReLU → MaxPool1d(2)`.

- **Block 3:** Conv1d(in=32, out=64, kernel=5, padding=2) → BatchNorm1d → ReLU → MaxPool1d(2).

The early layers use larger kernels (7) to capture broad temporal structure such as the overall envelope of anomaly shapes, while the final layer uses a smaller kernel (5) to refine local features. The channel count doubles at each block (16 → 32 → 64), progressively expanding the network’s feature representation capacity.

The network utilizes a **Regional Pooling layer** (AdaptiveAvgPool1d(4)) instead of Global Average Pooling. This preserves 4 distinct temporal bins, allowing the network to recognize the temporal topography of shapes—for example, distinguishing the two separate peaks of an M shape from a solid bell_sq block, since the averaged values in each bin differ between these shapes.

This flattened vector of size 256 (64 × 4) is then passed through a classifier head: Flatten() → Dropout(0.4) → Linear(256, 32) → ReLU → Dropout(0.2) → Linear(32, 7). The total model footprint is **22,727 trainable parameters**.

5.3 Training Loop (train.py)

The training script establishes the complete PyTorch training ecosystem with GPU acceleration and comprehensive logging:

- **Data Split:** The training set is further divided 80/20 into train/validation subsets using a random permutation for early stopping monitoring.
- **Class Imbalance Handling:** Computes inverse frequency weights $w_c = \frac{N_{\text{total}}}{C \cdot N_c}$ and passes them to the CrossEntropyLoss criterion, forcing the optimizer to treat rare anomaly shapes with the same importance as the dominant normal background.
- **Optimization:** Adam optimizer with initial learning rate 10^{-3} .
- **Learning Rate Scheduling:** ReduceLROnPlateau (patience 2, factor 0.5) monitors validation loss and halves the learning rate when progress stalls.
- **Early Stopping:** Patience of 5 epochs. The best model weights are saved only when validation loss improves.
- **Logging:** A comprehensive .txt log and an interactive Plotly training curves chart (HTML + PNG) are generated.

5.3.1 Training Results

The model was trained on a 10M-point synthetic dataset (249,985 windows) on an NVIDIA GeForce RTX 5060 Laptop GPU with CUDA 12.8:

- **Convergence:** 82 epochs, 9 minutes 5 seconds. Learning rate decayed from 10^{-3} to 8×10^{-6} across 7 reduction steps.
- **Final losses:** Best validation loss = 0.2494, test loss = 0.3028.
- **Test accuracy:** 93.4% per-window accuracy (62,485 windows). Event-level accuracy is significantly higher due to overlapping window accumulation during inference.

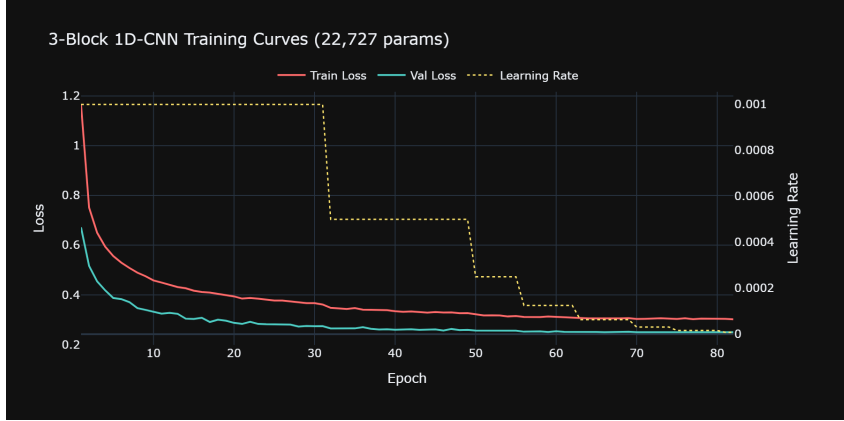


Figure 4: Training and validation loss curves with learning rate schedule. The validation loss remains below the training loss due to class-weighted cross-entropy inflating the training loss for minority anomaly classes.

5.4 Architecture Search

The final architecture was selected through an empirical search over several candidate configurations. All variants were trained on the same 10M-point dataset and evaluated on both per-window test accuracy and event-level inference over a 100,000-point segment containing 77 anomaly events:

- **Depth vs. Width:** Reducing the number of convolutional blocks below 3 degrades performance unless channel width is increased. A 2-block model with narrow channels (16→32) lacked sufficient representational capacity, while a 2-block model with wider channels (32→64) recovered most accuracy.
- **Diminishing returns of depth:** Blocks 4 and 5 contributed negligible accuracy gains while inflating the parameter count by 12 $\times$.
- **Training speed:** Tightening early stopping patience from 10 to 5 and LR scheduler patience from 4 to 2, combined with doubling the batch size to 128, reduced training time without affecting convergence.

Table 1: Comparison of 1D-CNN architecture variants. Event-level metrics are computed over 77 anomaly events in a 100K-point inference segment. All models use `AdaptiveAvgPool1d(4)` regional pooling.

Variant	Channels	Params	Train Time	Event Acc.	Mean Conf.
2-block narrow	16-32	3,719	8 min 12 s	88.3%	66.8%
2-block wide	32-64	19,207	5 min 49 s	98.7%	79.7%
3-block	16-32-64	22,727	5 min 45 s	100%	90.2%
4-block	16-32-64-64	35,207	7 min 18 s	100%	90.2%
5-block	32-64-128-128-256	270,725	7 min 58 s	100%	91.9%

The 3-block architecture was selected as the final model, achieving perfect event-level classification with 12 $\times$ fewer parameters than the 5-block architecture and the fastest training time.

5.5 Inference and Evaluation (visualize_inference.py)

The inference script evaluates the trained model on a continuous segment of the dataset using a fine-grained sliding window (stride 10, window 512). Two key design decisions:

1. **Consistent Normalization:** The inference script loads the global σ_{train} from `normalization_stats.json` and applies the same per-window centering strategy.
2. **Overlapping Window Accumulation:** The softmax probability vector is **accumulated** across all points covered by each window. With stride 10 and window 512, each point is covered by up to 51 overlapping windows. The final per-point class probability is the average:

$$P_{\text{final}}(c \mid t) = \frac{1}{|\mathcal{W}_t|} \sum_{w \in \mathcal{W}_t} P_w(c) \quad (8)$$

where $\mathcal{W}_t$ is the set of all windows covering time step t . This ensemble-like averaging significantly reduces prediction noise.



Figure 5: Multi-Class 1D-CNN Confidence Probability mapped against Simulated Gamma Noise

5.5.1 Inference Results

The model was evaluated on a 1,000,000-point segment containing 777 anomaly events:

Table 2: Event-level inference results over 777 anomaly events. Detection rate indicates whether the model identified the region as any anomaly type; exact classification requires the correct class.

Metric	Value
Total events	777
Detection rate (any anomaly)	100.0% (777/777)
Exact classification accuracy	98.3% (764/777)
Mean confidence	88.8%
Inference throughput	1,421 windows/sec
Inference time (1M points)	70 s

Table 3: Per-class Precision, Recall, and F1-Score at event-level granularity.

Class	Support	Precision	Recall	F1-Score
bell	128	0.992	0.992	0.992
bell_sq	136	0.963	0.956	0.959
fast_ascend	129	1.000	0.992	0.996
fast_descend	116	0.983	0.991	0.987
M	126	0.962	0.992	0.977
square	142	1.000	0.979	0.989
Macro Average		0.983	0.984	0.984

The 13 misclassifications are concentrated in the `bell_sq` class (6 errors), which is the most morphologically ambiguous shape—its flat-topped profile can be confused with `square`, and its rounded shoulders can resemble `M` at high deformation levels.

5.6 Robustness Analysis (`robustness_test.py`)

To assess the model’s sensitivity to degraded signal conditions, a robustness test independently sweeps two axes while holding the other at its training baseline:

1. **Background noise intensity:** The standard deviation of the high-frequency sensor noise was scaled from $1\times$ to $3\times$ the baseline value ($\sigma = 2.70$ gamma dose units).
2. **Anomaly shape deformation:** The variance parameter of the anomaly generator was scaled from $1\times$ to $8\times$ the baseline value (0.04), controlling how much each anomaly instance deviates from its canonical template shape.

Each configuration generated a test dataset with 50 anomaly events using a fixed random seed to ensure identical anomaly positions and template selections across all runs.

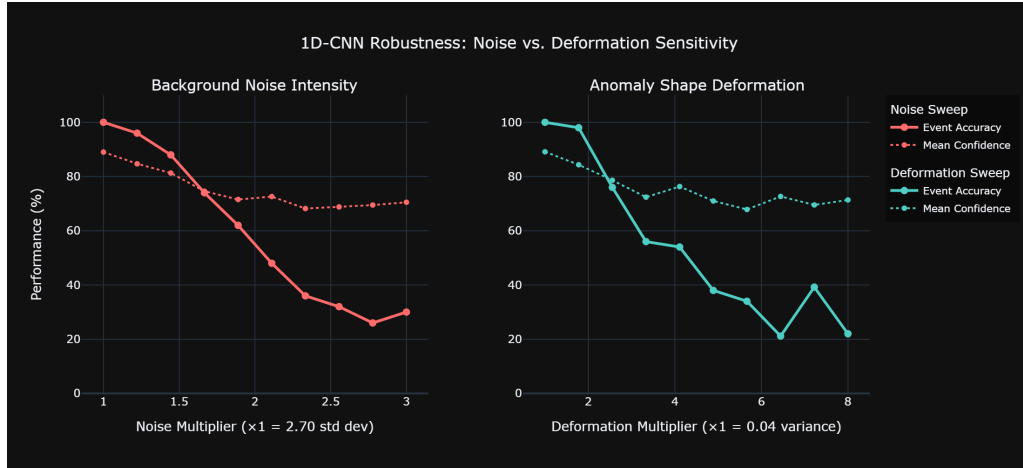


Figure 6: Model robustness under increasing noise (left) and shape deformation (right). Event accuracy (solid) and mean confidence (dashed) are shown. The model is substantially more sensitive to shape deformation than to background noise.